

LangGraph 技术调研：合同审查全链路方案

目录

- **LangGraph 技术调研：合同审查全链路方案**
 - 一、LangGraph 简介
 - 二、环境安装
 - 2.1 基础依赖
 - 2.2 完整依赖（合同审查场景）
 - 2.3 验证安装
 - 三、核心概念
 - 3.1 StateGraph
 - 3.2 节点（Node）
 - 3.3 边（Edge）与条件路由
 - 四、合同审查全链路 Demo 方案
 - 4.1 整体架构
 - 4.2 State 定义
 - 4.3 节点实现
 - Step 1：文档上传与格式识别
 - Step 2：文档解析
 - Step 3：章节切分
 - Step 4：向量化 & 文档召回
 - Step 5：LLM 审核
 - Step 6：结构化批注生成
 - Step 7：人工复核节点（Human-in-the-loop）

- Step 8: 最终报告生成
- 4.4 图组装
- 4.5 运行示例
- 五、关键设计决策
 - 5.1 章节切分策略对比
 - 5.2 召回增强方案
 - 5.3 LangGraph Streaming 接入前端
 - 5.4 持久化与并发
- 六、目录结构建议
- 七、参考资料

LangGraph 技术调研：合同审查全链路方案

一、LangGraph 简介

LangGraph 是 LangChain 生态下的有状态、多角色工作流编排框架，专为构建需要循环、条件分支、人工介入的复杂 LLM 应用而设计。

核心设计理念：

- **图结构编排**：以有向图（DAG/循环图）表达工作流，节点是函数，边是路由逻辑
- **持久化状态**：内置 Checkpointer 机制，支持断点续跑、多轮对话状态复用
- **Human-in-the-loop**：原生支持在任意节点暂停等待人工确认
- **Streaming**：支持 Token 级、节点级事件流式输出

与 LangChain Chain 的区别：Chain 是线性的，LangGraph 支持循环和条件跳转，适合审核-修改-再审核这类迭代流程。

二、环境安装

2.1 基础依赖

```
</> Bash |  
  
1 pip install langgraph==0.2.x  
2 pip install langchain-core langchain-openai  
3 # 文档解析  
4 pip install python-docx pypdf  
5 # 向量检索  
6 pip install faiss-cpu langchain-community  
7 # 可选：持久化 checkpointer  
8 pip install langgraph-checkpoint-sqlite
```

2.2 完整依赖（合同审查场景）

```
</> Bash |  
  
1 pip install \  
2   langgraph \  
3   langchain-core \  
4   langchain-openai \  
5   python-docx \  
6   pypdf \  
7   faiss-cpu \  
8   langchain-community \  
9   pydantic \  
10  langgraph-checkpoint-sqlite
```

2.3 验证安装

</>

Python

```
1 import langgraph
2 print(langgraph.__version__) # 0.2.x
```

三、核心概念

3.1 StateGraph

</>

Python

```
1 from langgraph.graph import StateGraph, END
2 from typing import TypedDict, Annotated
3 import operator
4
5 class ContractState(TypedDict):
6     file_path: str # 上传文件路径
7     raw_text: str # 原始文本
8     sections: list[dict] # 章节切片
9     retrieved_clauses: list[dict] # 召回条款
10    review_results: list[dict] # 审核结果
11    annotations: list[dict] # 结构化批注
12    messages: Annotated[list, operator.add] # 消息历史（追加语义）
13    status: str # 当前流程状态
```

3.2 节点 (Node)

节点是纯函数，接收 State，返回 State 的部分更新：

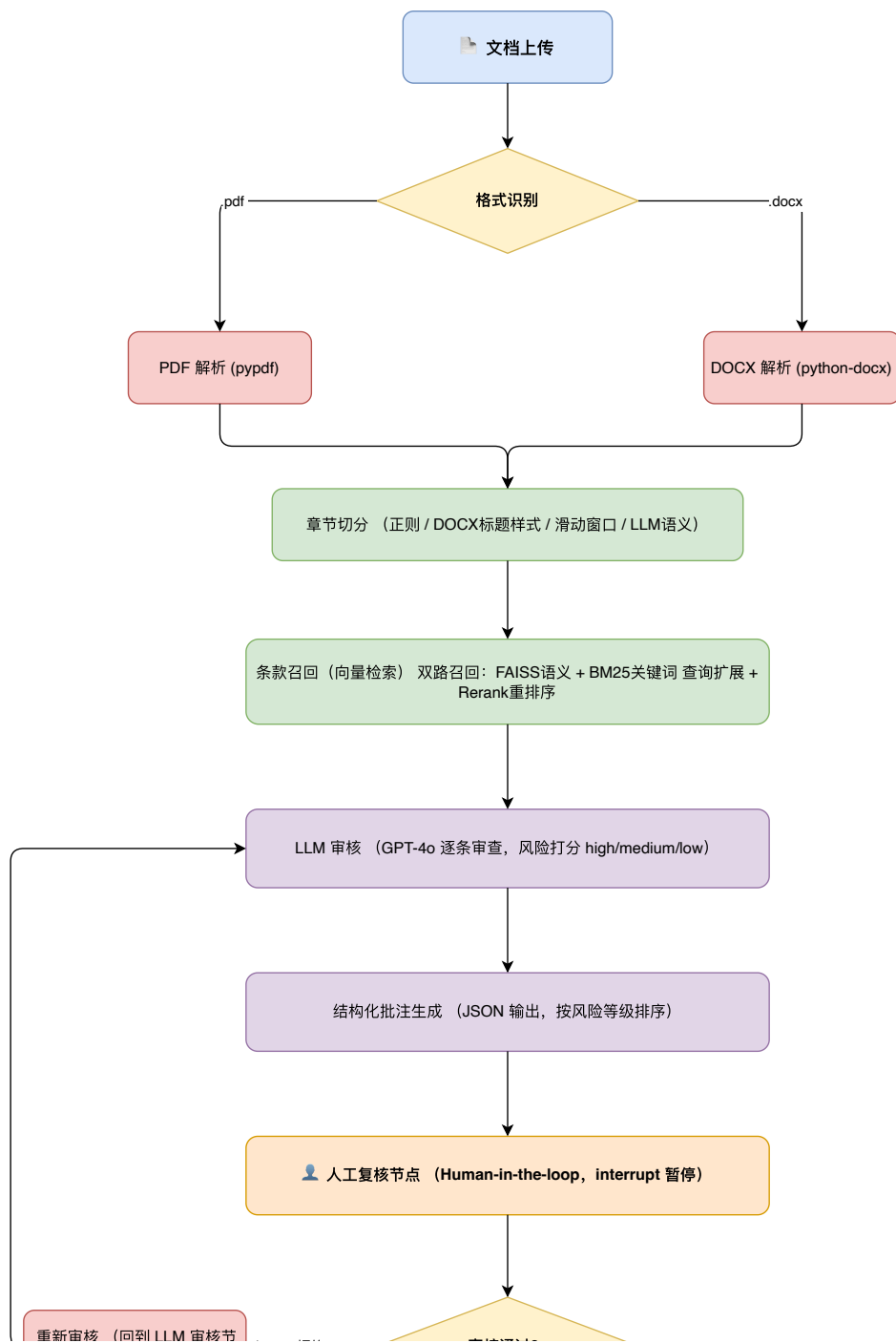
```
</> Python |  
  
1 def parse_document(state: ContractState) -> dict:  
2     # 处理逻辑  
3     return {"raw_text": parsed_text, "status": "parsed"}
```

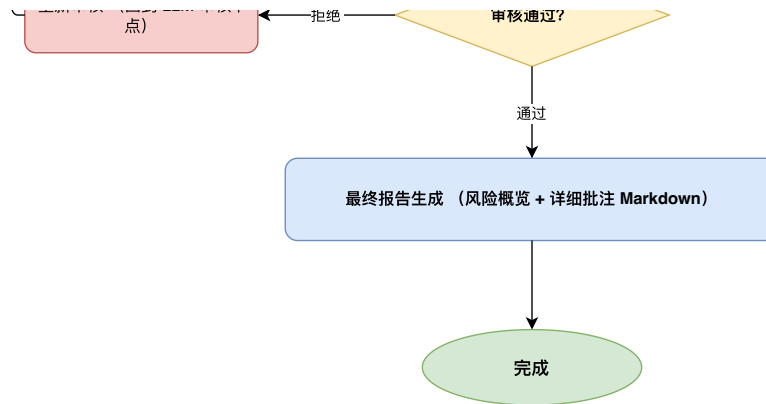
3.3 边 (Edge) 与条件路由

```
</> Python |  
  
1 # 固定边  
2 graph.add_edge("parse", "split")  
3  
4 # 条件边 (路由函数)  
5 def route_by_format(state: ContractState) -> str:  
6     if state["file_path"].endswith(".pdf"):  
7         return "parse_pdf"  
8     return "parse_docx"  
9  
10 graph.add_conditional_edges("upload", route_by_format, {  
11     "parse_pdf": "parse_pdf",  
12     "parse_docx": "parse_docx",  
13 })
```

四、合同审查全链路 Demo 方案

4.1 整体架构





4.2 State 定义

</>

Python

```
1 from typing import TypedDict, Annotated, List, Optional
2 import operator
3
4 class ReviewAnnotation(TypedDict):
5     clause_id: str      # 条款编号
6     clause_text: str    # 原文
7     risk_level: str     # high/medium/low
8     issue: str          # 问题描述
9     suggestion: str     # 修改建议
10    legal_basis: str     # 法律依据
11
12 class ContractState(TypedDict):
13     file_path: str
14     file_type: str      # pdf / docx
15     raw_text: str
16     sections: List[dict] # [{id, title, content}]
```



```
17     retrieved_clauses: List[dict]          # 召回的相关条款
18     review_results: List[ReviewAnnotation] # 审核结果
19     human_feedback: Optional[str]         # 人工反馈
20     final_report: Optional[str]           # 最终报告
21     messages: Annotated[list, operator.add]
22     status: str
```

4.3 节点实现

Step 1: 文档上传与格式识别

</>

Python

```
1 import os
2
3 def upload_and_detect(state: ContractState) -> dict:
4     file_path = state["file_path"]
5     ext = os.path.splitext(file_path)[1].lower()
6     file_type = "pdf" if ext == ".pdf" else "docx"
7     return {"file_type": file_type, "status": "detected"}
```

Step 2: 文档解析

</>

Python

```
1 from pypdf import PdfReader
2 from docx import Document
3
4 def parse_pdf(state: ContractState) -> dict:
```

```

5     reader = PdfReader(state["file_path"])
6     text = "\n".join(page.extract_text() or "" for page in reader.pages)
7     return {"raw_text": text, "status": "parsed"}
8
9 def parse_docx(state: ContractState) -> dict:
10     doc = Document(state["file_path"])
11     # 保留段落结构
12     paragraphs = []
13     for para in doc.paragraphs:
14         if para.text.strip():
15             paragraphs.append({
16                 "style": para.style.name,
17                 "text": para.text.strip()
18             })
19     raw_text = "\n".join(p["text"] for p in paragraphs)
20     return {"raw_text": raw_text, "status": "parsed"}

```

Step 3: 章节切分

</>

Python

```

1 import re
2 from typing import List
3
4 def split_sections(state: ContractState) -> dict:
5     """
6     按中文合同章节规律切分：
7     - 第X条、X.X、一、二、三 等标题模式
8     """

```

```

9     text = state["raw_text"]
10    # 匹配: 第X条 / X.Y / 一、二、 等条款编号
11    pattern = r'(?=(?:第[一二三四五六七八九十百\d]+条|[一二三四五六七八九十]+、|\d+\.\d+\s))'
12    parts = re.split(pattern, text)
13
14    sections = []
15    for i, part in enumerate(parts):
16        part = part.strip()
17        if not part:
18            continue
19        # 提取标题 (第一行)
20        lines = part.split('\n', 1)
21        title = lines[0].strip()
22        content = lines[1].strip() if len(lines) > 1 else part
23        sections.append({
24            "id": f"sec_{i:03d}",
25            "title": title,
26            "content": content,
27            "full_text": part
28        })
29
30    return {"sections": sections, "status": "split"}

```

Step 4: 向量化 & 文档召回

</>

Python

```

1 from langchain_community.vectorstores import FAISS
2 from langchain_openai import OpenAIEmbeddings

```

```
3 from langchain_core.documents import Document as LCDoc
4
5 # 初始化（复用，避免重复创建）
6 embeddings = OpenAIEmbeddings()
7
8 def build_index_and_retrieve(state: ContractState) -> dict:
9     sections = state["sections"]
10
11     # 构建 LangChain Document 列表
12     docs = [
13         LCDoc(
14             page_content=sec["full_text"],
15             metadata={"id": sec["id"], "title": sec["title"]}
16         )
17         for sec in sections
18     ]
19
20     # 建立 FAISS 索引
21     vectorstore = FAISS.from_documents(docs, embeddings)
22
23     # 召回与合同风险相关的核心条款
24     risk_queries = [
25         "违约责任",
26         "付款条件和期限",
27         "争议解决方式",
28         "保密条款",
29         "知识产权归属",
30         "合同解除条件",
31     ]
```

```
32
33     retrieved = []
34     seen_ids = set()
35     for query in risk_queries:
36         results = vectorstore.similarity_search(query, k=2)
37         for doc in results:
38             sec_id = doc.metadata["id"]
39             if sec_id not in seen_ids:
40                 seen_ids.add(sec_id)
41                 retrieved.append({
42                     "id": sec_id,
43                     "title": doc.metadata["title"],
44                     "content": doc.page_content,
45                     "matched_query": query
46                 })
47
48     return {"retrieved_clauses": retrieved, "status": "retrieved"}
```

Step 5: LLM 审核

</>

Python

```
1 from langchain_openai import ChatOpenAI
2 from langchain_core.messages import SystemMessage, HumanMessage
3 import json
4
5 llm = ChatOpenAI(model="gpt-4o", temperature=0)
6
7 REVIEW_SYSTEM_PROMPT = """你是一名资深合同律师，专门审查合同条款风险。
```

```
8 对每个条款进行分析，输出 JSON 格式结果：
9 {
10     "clause_id": "条款编号",
11     "clause_text": "原文摘要",
12     "risk_level": "high|medium|low|none",
13     "issue": "存在的问题（无问题则为空字符串）",
14     "suggestion": "修改建议（无建议则为空字符串）",
15     "legal_basis": "相关法律依据"
16 }
17 只输出 JSON，不要有额外说明。"""
18
19 def review_clauses(state: ContractState) -> dict:
20     clauses = state["retrieved_clauses"]
21     results = []
22
23     for clause in clauses:
24         prompt = f"条款ID: {clause['id']}\n条款标题: {clause['title']}\n条款内容:\n{clause['content'][:800]}"
25
26         response = llm.invoke([
27             SystemMessage(content=REVIEW_SYSTEM_PROMPT),
28             HumanMessage(content=prompt)
29         ])
30
31         try:
32             result = json.loads(response.content)
33             results.append(result)
34         except json.JSONDecodeError:
35             # 降级处理
36             results.append({
```

```
37         "clause_id": clause["id"],
38         "clause_text": clause["content"][:100],
39         "risk_level": "low",
40         "issue": "",
41         "suggestion": "",
42         "legal_basis": ""
43     })
44
45     return {"review_results": results, "status": "reviewed"}
```

Step 6: 结构化批注生成

</>

Python

```
1 def generate_annotations(state: ContractState) -> dict:
2     results = state["review_results"]
3
4     # 过滤出有问题的条款，按风险等级排序
5     risk_order = {"high": 0, "medium": 1, "low": 2, "none": 3}
6     annotations = [r for r in results if r.get("risk_level") in ("high", "medium", "low")]
7     annotations.sort(key=lambda x: risk_order.get(x.get("risk_level", "none"), 3))
8
9     return {"annotations": annotations, "status": "annotated"}
```

Step 7: 人工复核节点 (Human-in-the-loop)

</>

Python

```
1 def human_review_node(state: ContractState) -> dict:
```

```
2 """
3 此节点会被 interrupt 暂停，等待人工输入。
4 实际部署时通过 graph.update_state() 传入 human_feedback。
5 """
6 # 这里只做 pass-through，中断由外部 interrupt 配置控制
7 feedback = state.get("human_feedback", "")
8 if feedback:
9     return {"status": "human_approved", "messages": [{"role": "human", "content": feedback}]}
10 return {"status": "pending_human"}
```

Step 8: 最终报告生成

</>

Python

```
1 def generate_report(state: ContractState) -> dict:
2     annotations = state["annotations"]
3
4     high_risks = [a for a in annotations if a["risk_level"] == "high"]
5     medium_risks = [a for a in annotations if a["risk_level"] == "medium"]
6     low_risks = [a for a in annotations if a["risk_level"] == "low"]
7
8     report_lines = [
9         "# 合同审查报告",
10        f"\n## 风险概览",
11        f"- 高风险条款: {len(high_risks)} 项",
12        f"- 中风险条款: {len(medium_risks)} 项",
13        f"- 低风险条款: {len(low_risks)} 项",
14        "\n## 详细批注",
15    ]
```



```

16
17     for anno in annotations:
18         report_lines += [
19             f"\n### [{anno['risk_level'].upper()}] {anno.get('clause_id', '')}",
20             f"**原文摘要**: {anno.get('clause_text', '')}",
21             f"**问题**: {anno.get('issue', '无')}",
22             f"**建议**: {anno.get('suggestion', '无')}",
23             f"**法律依据**: {anno.get('legal_basis', '无')}",
24         ]
25
26     report = "\n".join(report_lines)
27     return {"final_report": report, "status": "done"}

```

4.4 图组装

</>

Python

```

1 from langgraph.graph import StateGraph, END
2 from langgraph.checkpoint.sqlite import SqliteSaver
3
4 def build_contract_review_graph():
5     graph = StateGraph(ContractState)
6
7     # 注册节点
8     graph.add_node("upload_detect", upload_and_detect)
9     graph.add_node("parse_pdf", parse_pdf)
10    graph.add_node("parse_docx", parse_docx)
11    graph.add_node("split_sections", split_sections)
12    graph.add_node("retrieve", build_index_and_retrieve)

```

```
13 graph.add_node("review", review_clauses)
14 graph.add_node("annotate", generate_annotations)
15 graph.add_node("human_review", human_review_node)
16 graph.add_node("report", generate_report)
17
18 # 入口
19 graph.set_entry_point("upload_detect")
20
21 # 条件路由: 根据文件类型选解析器
22 graph.add_conditional_edges(
23     "upload_detect",
24     lambda s: "parse_pdf" if s["file_type"] == "pdf" else "parse_docx",
25     {"parse_pdf": "parse_pdf", "parse_docx": "parse_docx"}
26 )
27
28 # 固定边
29 graph.add_edge("parse_pdf", "split_sections")
30 graph.add_edge("parse_docx", "split_sections")
31 graph.add_edge("split_sections", "retrieve")
32 graph.add_edge("retrieve", "review")
33 graph.add_edge("review", "annotate")
34 graph.add_edge("annotate", "human_review")
35
36 # 人工审核后路由: approved → report, rejected → review (重审)
37 graph.add_conditional_edges(
38     "human_review",
39     lambda s: "report" if s["status"] == "human_approved" else "review",
40     {"report": "report", "review": "review"}
41 )
```

```

42
43     graph.add_edge("report", END)
44
45     # 持久化 checkpoint (支持断点续跑)
46     checkpointer = SqliteSaver.from_conn_string("contract_review.db")
47     return graph.compile(
48         checkpointer=checkpointer,
49         interrupt_before=["human_review"] # 到达 human_review 节点前暂停
50     )

```

4.5 运行示例

</>

Python

```

1 app = build_contract_review_graph()
2
3 # 第一阶段: 运行到 human_review 暂停
4 config = {"configurable": {"thread_id": "contract_001"}}
5 initial_state = {
6     "file_path": "./contracts/sample_contract.docx",
7     "messages": [],
8     "status": "start"
9 }
10
11 # 流式执行, 自动在 human_review 前暂停
12 for event in app.stream(initial_state, config):
13     node_name = list(event.keys())[0]
14     print(f"✅ 节点完成: {node_name} → status={event[node_name].get('status', '')}")
15

```

```
16 # 查看当前批注结果
17 snapshot = app.get_state(config)
18 annotations = snapshot.values.get("annotations", [])
19 print(f"\n共发现 {len(annotations)} 条风险批注")
20 for a in annotations[:3]:
21     print(f" [{a['risk_level']}] {a.get('clause_id')}: {a.get('issue')}")
22
23 # 第二阶段：人工介入，传入反馈后继续
24 app.update_state(config, {"human_feedback": "已确认，批注内容准确，请生成报告"})
25 for event in app.stream(None, config):
26     node_name = list(event.keys())[0]
27     print(f"✅ 节点完成: {node_name}")
28
29 # 获取最终报告
30 final_state = app.get_state(config)
31 print(final_state.values["final_report"])
```

五、关键设计决策

5.1 章节切分策略对比

策略	适用场景	优点	缺点
正则按条款编号切分	标准格式合同	语义完整、边界清晰	对非标准格式失效
固定 Token 滑动窗口	格式混乱文本	通用性强	可能截断语义
标题样式识别 (DOCX)	Word 格式合同	利用文档结构	仅限 DOCX

5	LLM 语义切分	复杂嵌套结构	语义最准确	成本高、慢
---	----------	--------	-------	-------

推荐：优先用正则 + DOCX 标题样式，fallback 到固定窗口。

5.2 召回增强方案

- 双路召回：向量语义召回 + BM25 关键词召回，取并集
- 查询扩展：对每个风险维度生成多个查询变体
- 重排序 (Rerank)：用 cross-encoder 对召回结果重排

5.3 LangGraph Streaming 接入前端

</> Python |

```
1 # 支持 SSE 流式推送节点进度到前端
2 async for event in app.astream_events(initial_state, config, version="v2"):
3     if event["event"] == "on_chain_end":
4         yield f"data: {json.dumps({'node': event['name'], 'status': 'done'})}\n\n"
```

5.4 持久化与并发

- 使用 SqliteSaver (本地单机) 或 PostgresSaver (生产多实例)
- thread_id 对应每个合同审查任务，支持同时处理多份合同
- Checkpointer 自动保存每个节点的状态快照，支持任意节点恢复

六、目录结构建议

</> Plain Text |

```
1 contract_review/
2 └─ graph/
3   └─ state.py          # ContractState 定义
4   └─ nodes.py          # 各节点函数
5   └─ router.py         # 条件路由逻辑
6   └─ builder.py        # 图组装入口
7 └─ parsers/
8   └─ pdf_parser.py     # pypdf 解析
9   └─ docx_parser.py    # python-docx 解析
10 └─ retrieval/
11   └─ splitter.py      # 章节切分
12   └─ vectorstore.py   # FAISS 索引管理
13 └─ review/
14   └─ reviewer.py      # LLM 审核逻辑
15   └─ prompts.py       # Prompt 模板
16 └─ output/
17   └─ reporter.py      # 报告生成
18 └─ main.py            # 入口
```

七、参考资料

- LangGraph 官方文档: <https://langchain-ai.github.io/langgraph/>
- LangGraph How-to Guides: <https://langchain-ai.github.io/langgraph/how-tos/>
- python-docx 文档: <https://python-docx.readthedocs.io/>
- pypdf 文档: <https://pypdf.readthedocs.io/>